AMRITA VISHWA VIDYAPEETHAM

22MAT230-Mathematics for Computing 4

Low-Rank Training of Neural Networks using SVD-Based Parameterization

Nimmagadda Kesav Satya Sai	CB.SC.U4AIE24236
Nakka Saampotth Maddileti	CB.SC.U4AIE24233
Gnana Vikas Sai Pabbathi	CB.SC.U4AIE24238
Vemula Poornachandra	CB.SC.U4AIE24258

Made with GAMMA

Introduction

- Modern deep neural networks are memory heavy ,computationally expensive and difficult to deploy on edge devices, mobile / embedded systems
- Weight matrices are already low-rank but the rank of the wieght matrix is not controlled during the training itself
- In this project we will be training the neural networks in such a way that directly singular vectors and singular values will be trained directly and pruned after every step such that only valuable eigen directions stay in the weight matrix. This helps reduce parameters

Made with GAMMA

Existing Approaches

Post-training SVD

- Decompose pretrained weights
- Accuracy loss at high compression

Nuclear norm regularization

- Requires SVD every iteration

Filter / channel pruning

- Architecture constraints are very important for this, if the network is rnn or resnet kind then it is very hard

Made with GAMMA

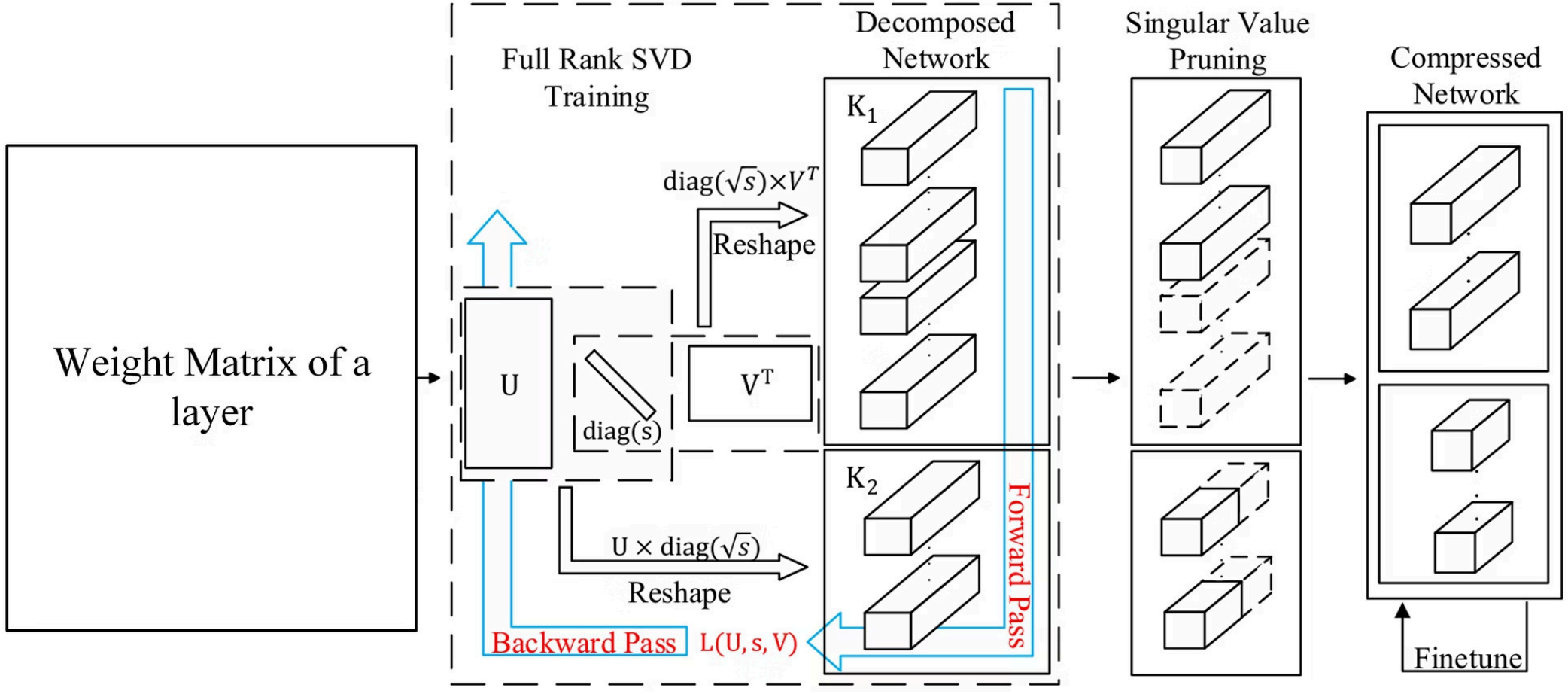
Our approach

- Represent each layer directly in SVD form
- Train singular vectors, singular values directly
- Enforce orthogonality on singular vectors and sparsity on singular values using regularizers to avoid exploding gradients and SVD functionalities

Made with GAMMA

Main Idea

For a DNN, the process goes like this



Made with GAMMA

Dataset Information

DATASET-1

Name: MNIST

Total Images: 70000

Training Images: 60000 (already separated by the publishers)

Testing Images: 10000

Number of classes: 10(0-9 digits)

Image Size: 28×28

Made with GAMMA

Dataset Information

DATASET-2

Name: Fashion-MNIST

Total Images: 70,000

Training Images: 60,000 (already separated by the publishers)

Testing Images: 10,000

Number of Classes: 10 (fashion product categories)

Image Size: 28 × 28 pixels

Made with GAMMA

Standard DNN Architecture

For both Datasets,

- **Input layer:**
  28 × 28 grayscale image will be flattened to a vector x (x belongs to R^784)
- **Hidden Layer 1:**
  Weight matrix W1 (W1 belongs to R^128×784)
  Activation: ReLU
- **Hidden Layer 2:**
  Weight matrix W2 (W2 belongs to R^64×128)
  Activation: ReLU
- **Output Layer:**
  Weight matrix W3 (W3 belongs to R^10×64)
  Activation: Softmax

Forward Pass

$$
\begin{aligned}
z_1 &= W_1 x + b_1, \quad a_1 = \mathrm{ReLU}(z_1) \\
z_2 &= W_2 a_1 + b_2, \quad a_2 = \mathrm{ReLU}(z_2) \\
z_3 &= W_3 a_2 + b_3, \quad \hat{y} = \mathrm{Softmax}(z_3)
\end{aligned}
$$

Made with GAMMA

Backpropagation in Standard DNN

Loss Function used:

Cross-Entropy Loss

$$\mathcal{L} = -\sum_{k=1}^{10} y_k \log(\hat{y}_k)$$

Weights are updated using gradient descent

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$

Limitations

- Large number of parameters
- High computational cost
- Redundant weights in fully connected layers

Made with GAMMA

SVD modified DNN Architecture

Fully connected weight matrices are often low-rank

Singular Value Decomposition (SVD) enables:

- Parameter reduction
- FLOPs reduction
- Model compression

- Instead of learning a full weight matrix W, each layer is parameterized directly using SVD components
- No SVD is performed during training
- U,S,V are initialized at the beginning and trained directly. But during forward pass we just multiply them in 2 layers

$$W = U\sqrt{\Sigma} \cdot \sqrt{\Sigma}V^T$$

Instead of Z=wx+b, we will be doing,

H=sqrt(s)*(V^T)*x

Z=U*sqrt(s)*H + b

Made with GAMMA

Loss Function in modified DNN

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda_{orth}\left(\|U^T U - I\|_F^2 + \|V^T V - I\|_F^2\right) + \lambda_{sparse}\frac{\|\Sigma\|_1}{\|\Sigma\|_2}$$

Overall Loss Function= Cross entropy loss + orthogonality loss + Hoyer Regularizer

Cross Entropy is the normal training loss

Orthogonality Loss is introduced to preserve the SVD properties

Sparsity is forced in this method using Hoyer regularizer so that rank will become less which leads to low parameters

Made with GAMMA

Backpropagation in SVD modified DNN

Weights are never trained directly, we backpropagate and train the U,S,V matrices directly from the feedback of loss function

The network trains:

- Left singular vectors, Singular values, Right singular vectors
- This enforces low-rank structure during training itself

Gradients are computed for loss function with respect to U,S,V(Respectively) and updated using gradient descent

Made with GAMMA

Rank Pruning

After convergence, effective rank is selected using energy criterion

$$\sum_{j \in \mathbb{K}} s_j^2 \leq e \sum_{i=1}^{r} s_i^2$$

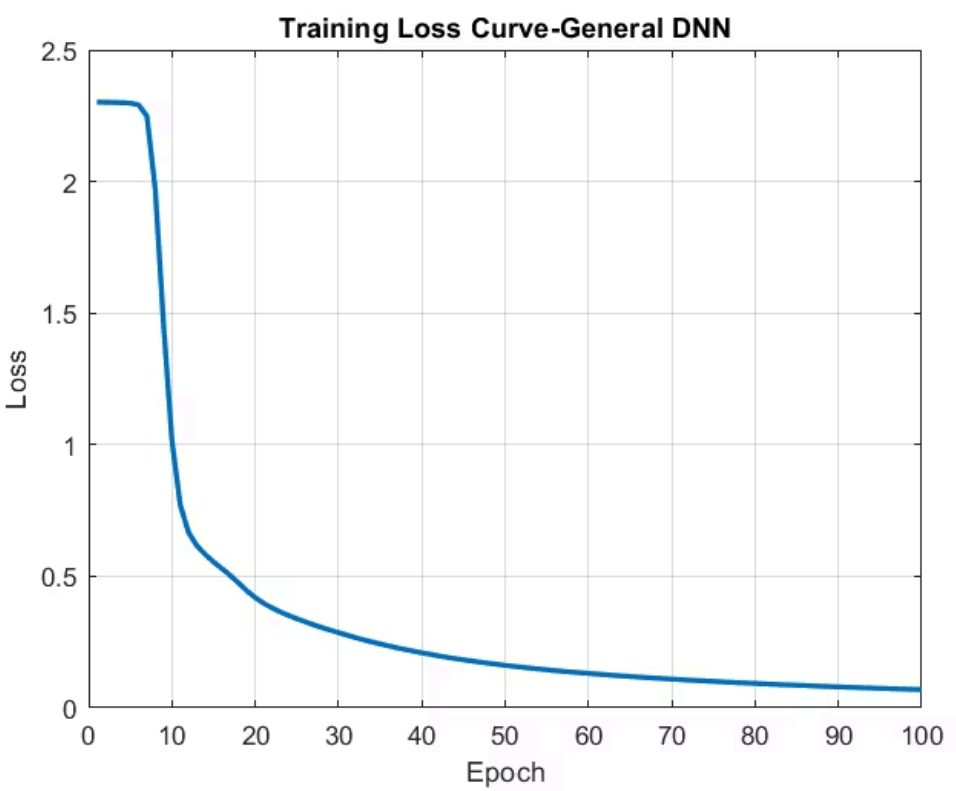
Corresponding columns of U and V are pruned accordingly

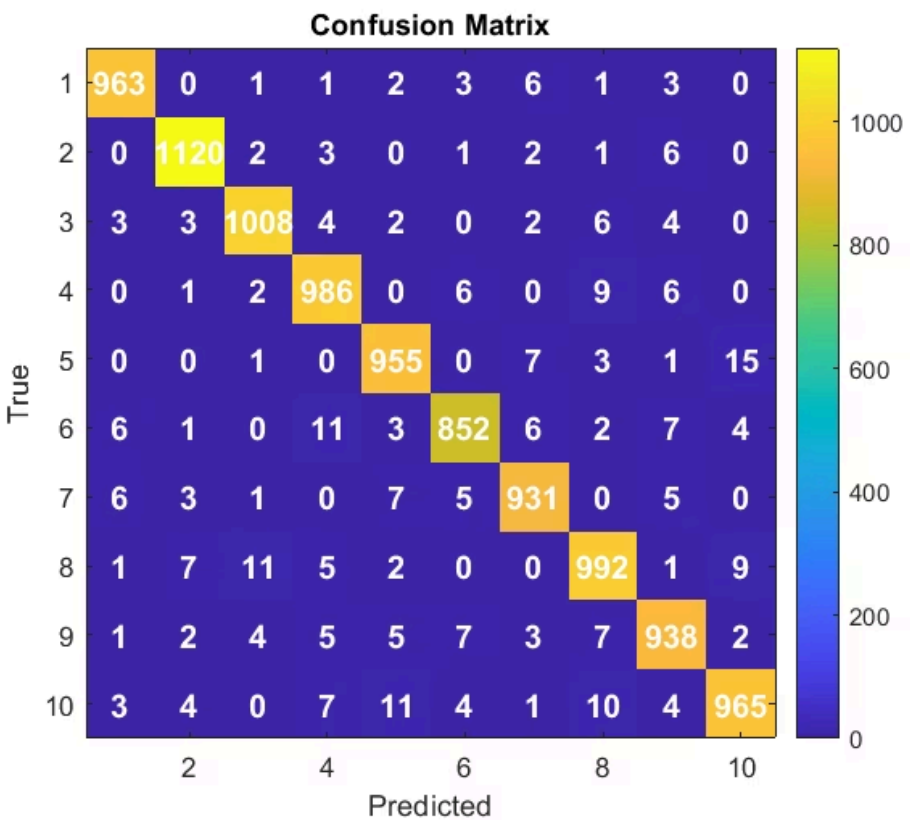
In the experiments performed, e=0.99

Made with GAMMA

Standard DNN Results (MNIST)

Accuracy: 97.10%

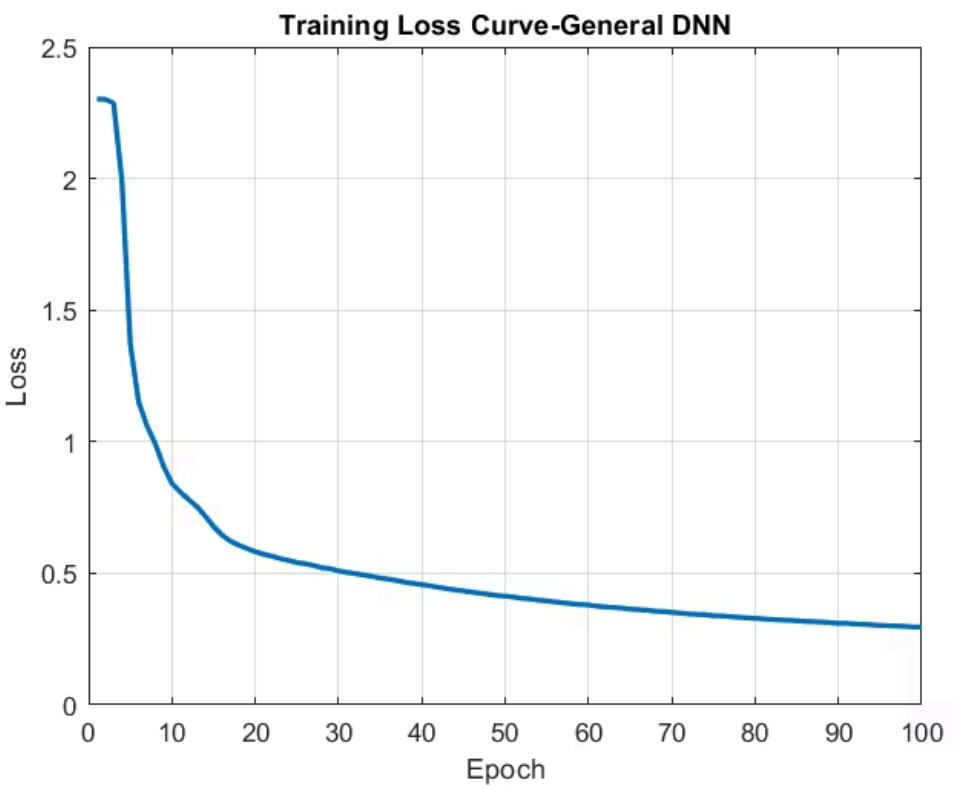


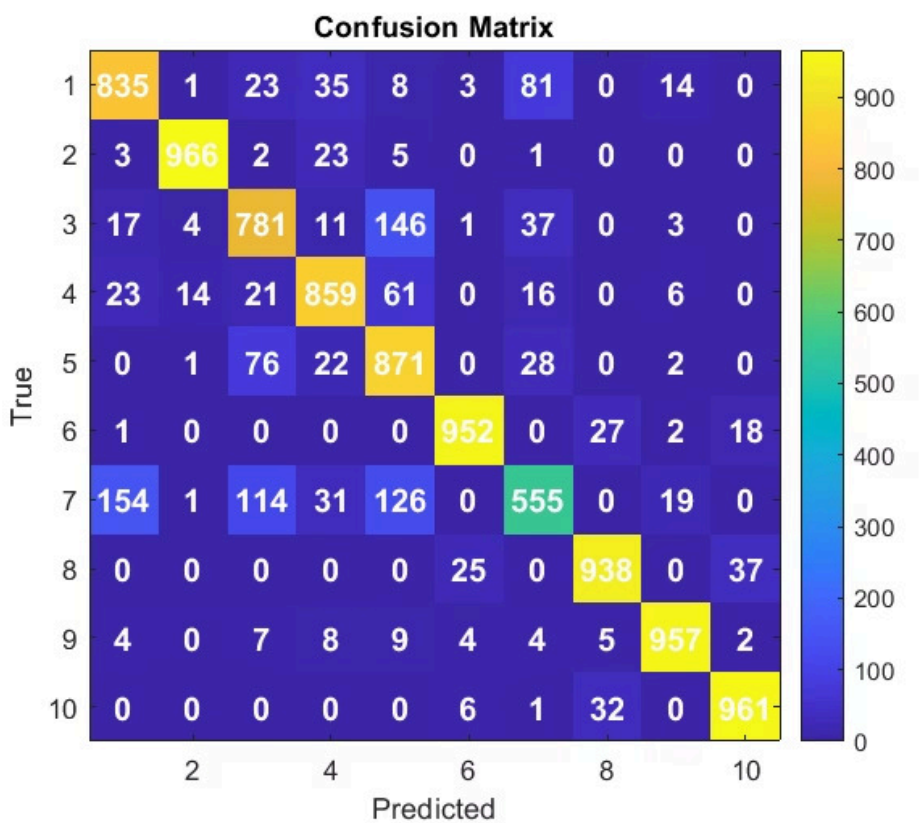


Made with GAMMA

Standard DNN results (Fashion-MNIST)

Accuracy: 86.75%

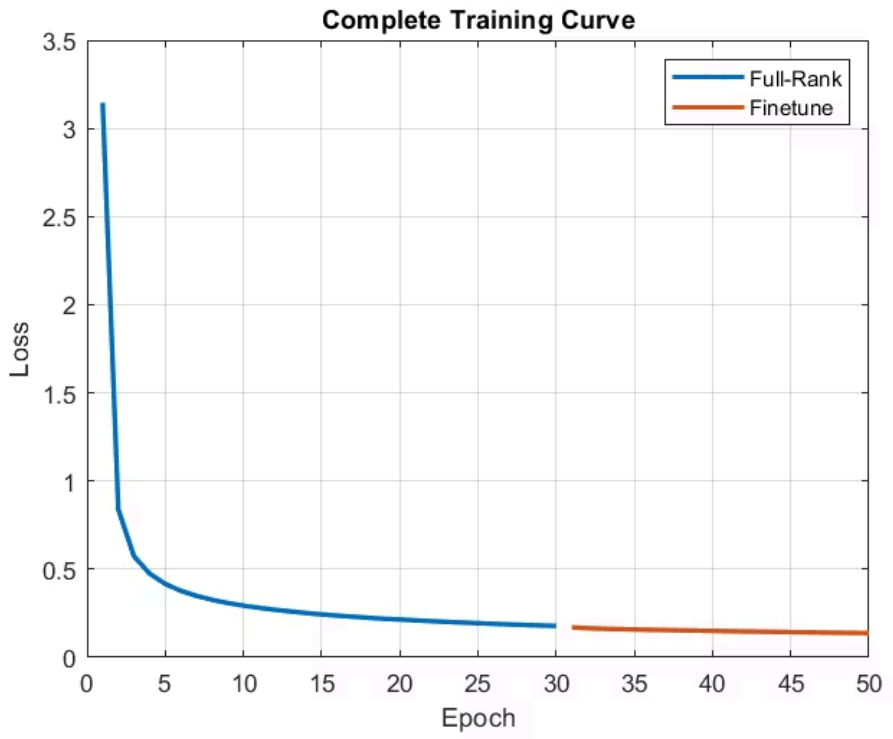


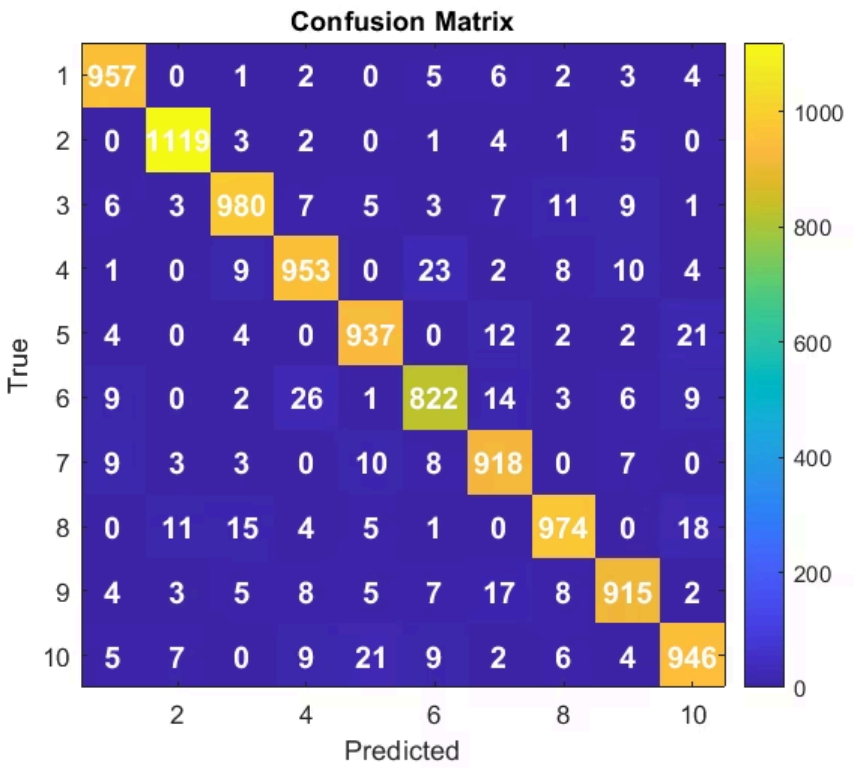


Made with GAMMA

SVD training result (MNIST)

Accuracy: 95.21%





Made with GAMMA

SVD training result (MNIST)

	Before Pruning	After Pruning
Rank of Weight matrices	[128 64 10]	[82 43 8]
Parameters	129966	83765
FLOPs	259528	167264

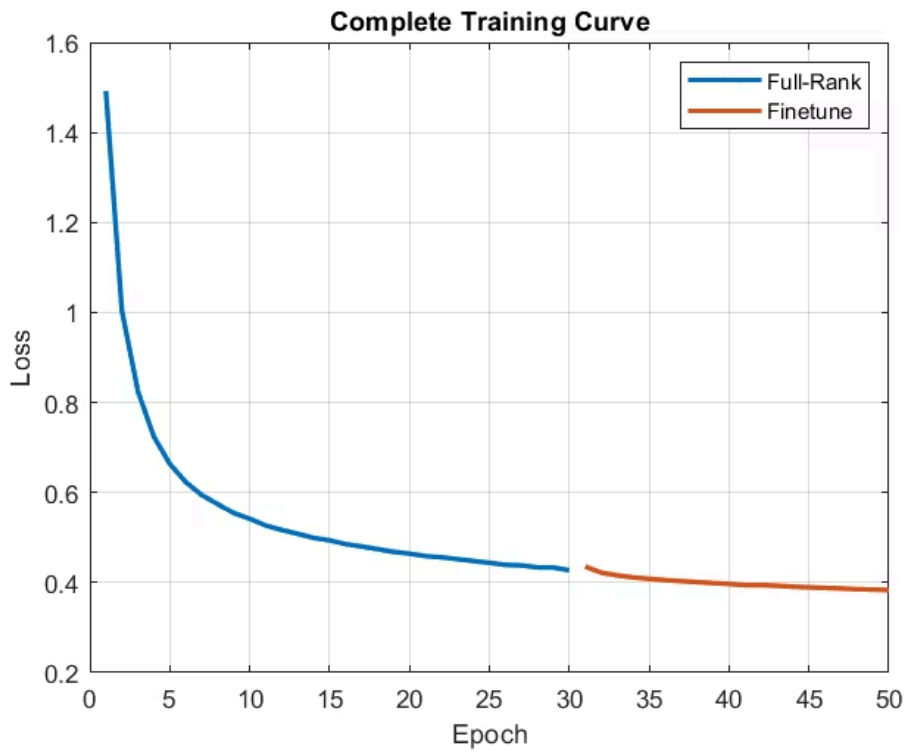
Parameters compression ratio=1.55x

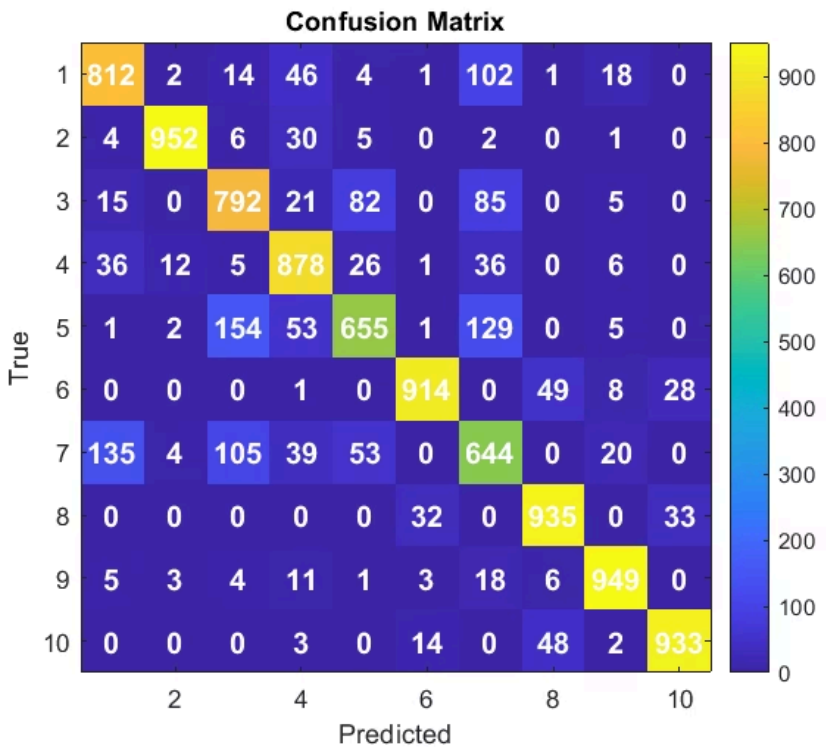
FLOPs reduction=35.55%

Made with GAMMA

SVD training result (Fashion MNIST)

Accuracy: 84.64%





Made with GAMMA

SVD training result (Fashion MNIST)

	Before Pruning	After Pruning
Rank of Weight matrices	[128 64 10]	[75 40 7]
Parameters	129966	76720
FLOPs	259528	153196

Parameters compression ratio=1.69x

FLOPs reduction=40.97%

Made with GAMMA

Experiments

Ran simulations changing learning rate, lambda_ortho parameters

Below is the table of the recorded accuracies at different parameters

LR

LR vs Lambda_ortho	1e-1	1e-2	1e-3	1e-4	1e-5	1e-6
1e-1	9.8% (Overfit)	93.03%(highly unstable)	93.37%	86.35%	64.07%	31.85%
1e-2	9.8% (Overfit)	9.8%(overfit)	93.41%	87.14%	64.26%	31.82%
1e-3	9.8% (Overfit)	9.8%(overfit)	94.86%	87.18%	64.27%	31.83%
1e-4	9.8% (Overfit)	93.75%	94.78%	87.18%	64.29%	31.83%
1e-5	9.8% (Overfit)	93.30%	94.80%	87.16%	64.29%	31.83%
1e-6	9.8% (Overfit)	92.81%	94.74%	87.81%	64.29%	31.83%

Made with GAMMA

Standard DNN training

Learning rate	Accuracy
1e-1	97.74%
1e-2	97.10%
1e-3	74.17%
1e-4	11.35%
1e-5	11.35%
1e-6	8.48%

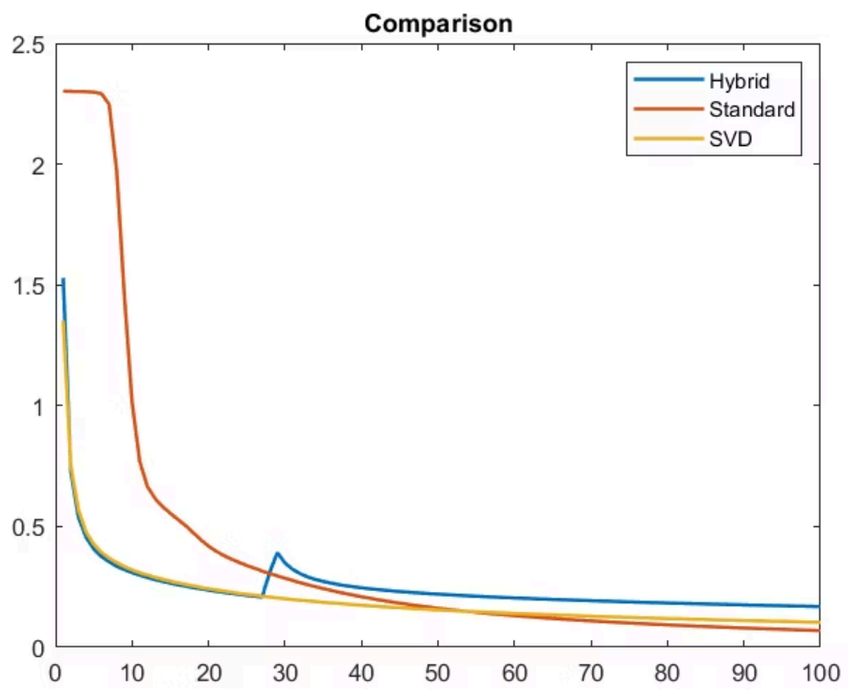
Made with GAMMA

Hybrid Version (based on loss difference from previous epoch)

Implemented Hybrid version of standard DNN and SVD-modified DNN.
Idea: In SVD-Modified DNN, the loss will drop fast but accuracy is low, in standard version, accuracy is high but slow convergence. So we will be training SVD-matrices directly until the loss is near to the optimal solution, then we will switch to standard training the whole weight matrix at a time. Loss function also changes simultaneously.

Result for MNIST dataset

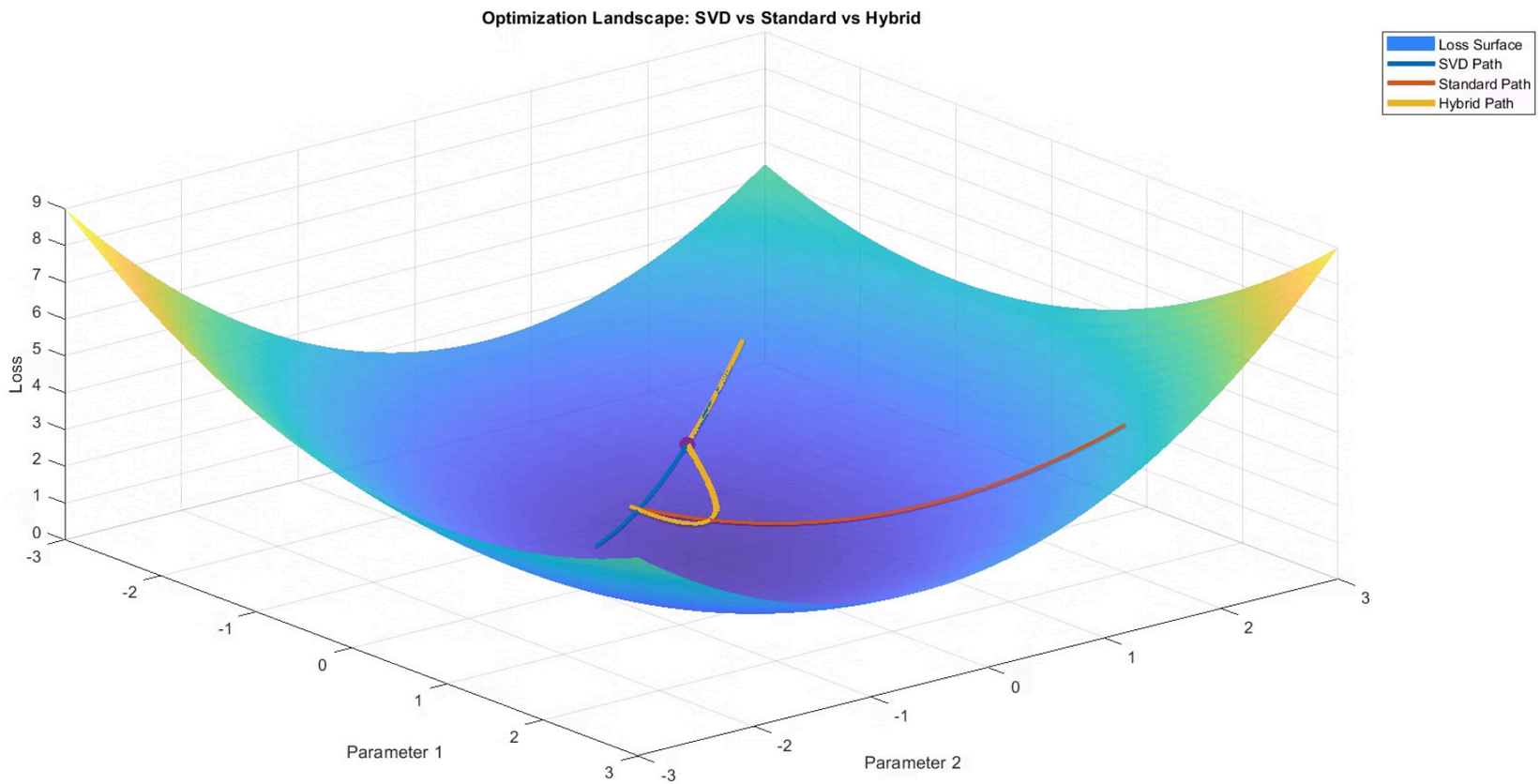
Accuracy: 94.73%



Made with GAMMA

Possible reasons for sudden spike in loss

- The spike happens maybe because the optimization landscape suddenly changes.
- Both standard DNN training and SVD-based training start from the same initialization but follow different optimization trajectories due to their different parameterizations.
- When the hybrid model switches from SVD parameterization to standard weight optimization, the optimizer transitions from one trajectory to another. This abrupt change in optimization direction can temporarily increase the loss, which appears as a spike in the training curve.

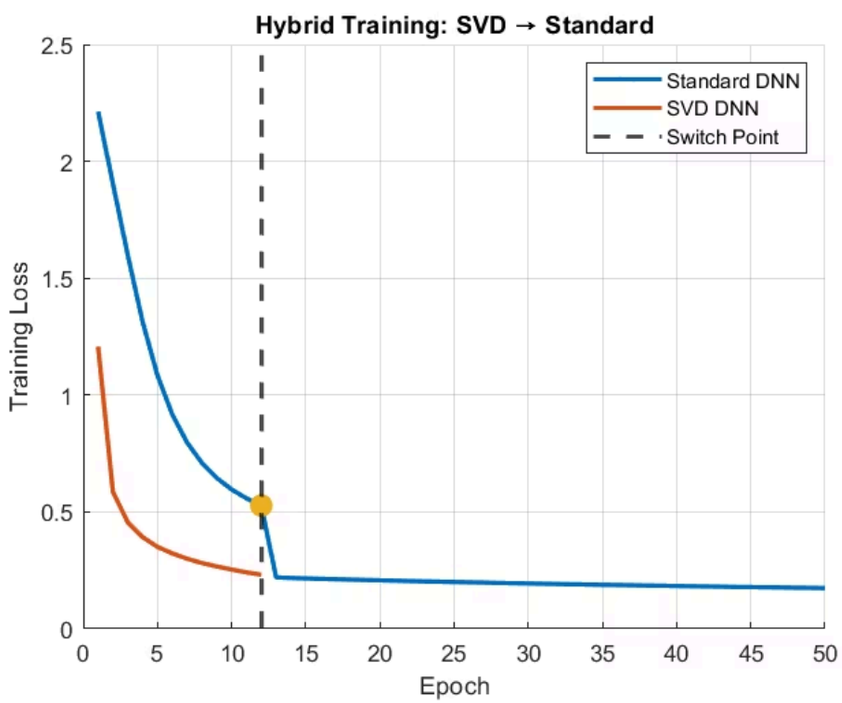


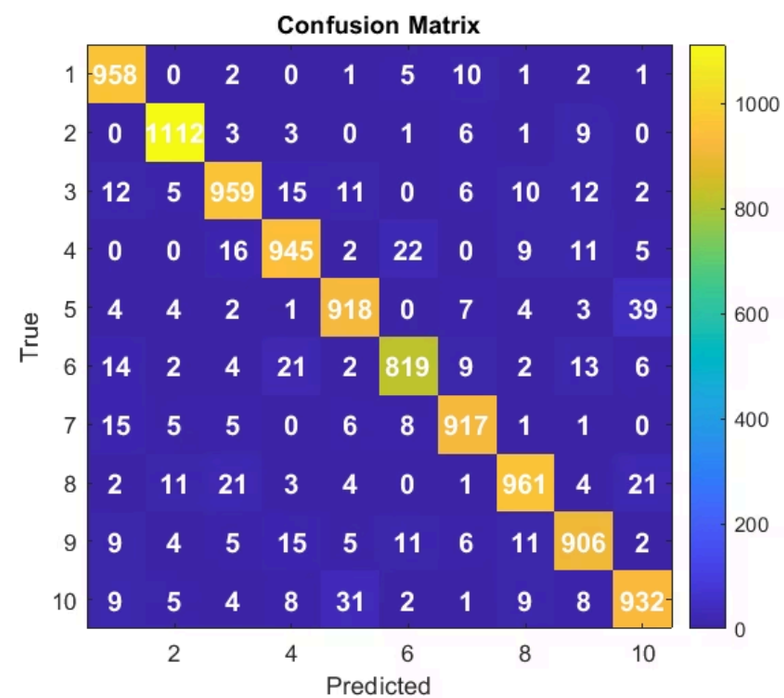
Made with GAMMA

Hybrid Version (based on loss equality of standard and svd training)

Implemented Hybrid version of standard DNN and SVD-modified DNN.
Idea: When the difference between the SVD loss and standard loss falls below a tolerance threshold, the training switches to standard DNN training using reconstructed weight matrices ($W = U \cdot S \cdot V^T$). This allows the network to fine-tune the full weight matrices and achieve higher final accuracy.

Dataset-MNIST
Accuracy: 94.27%



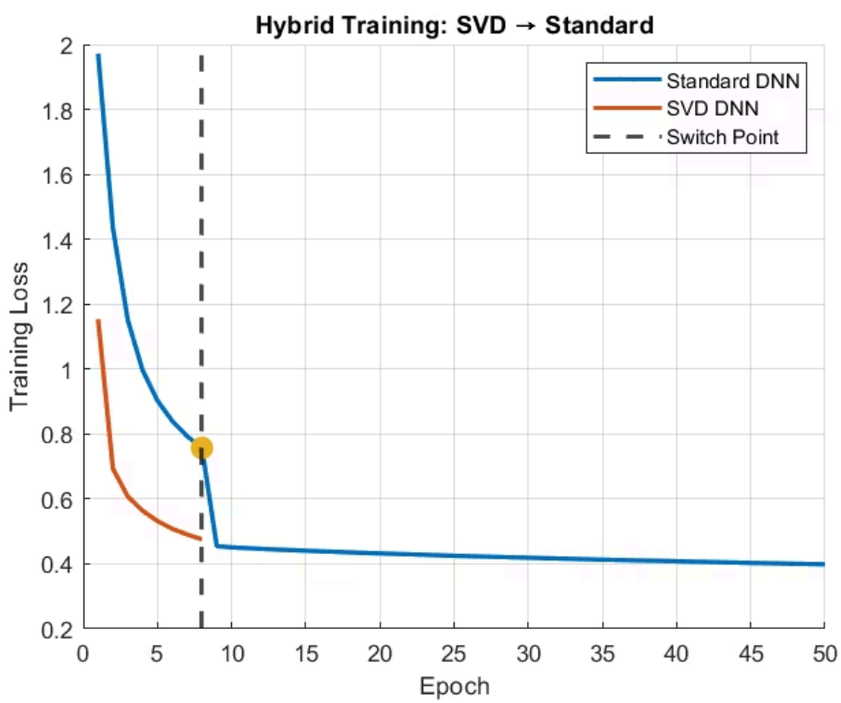


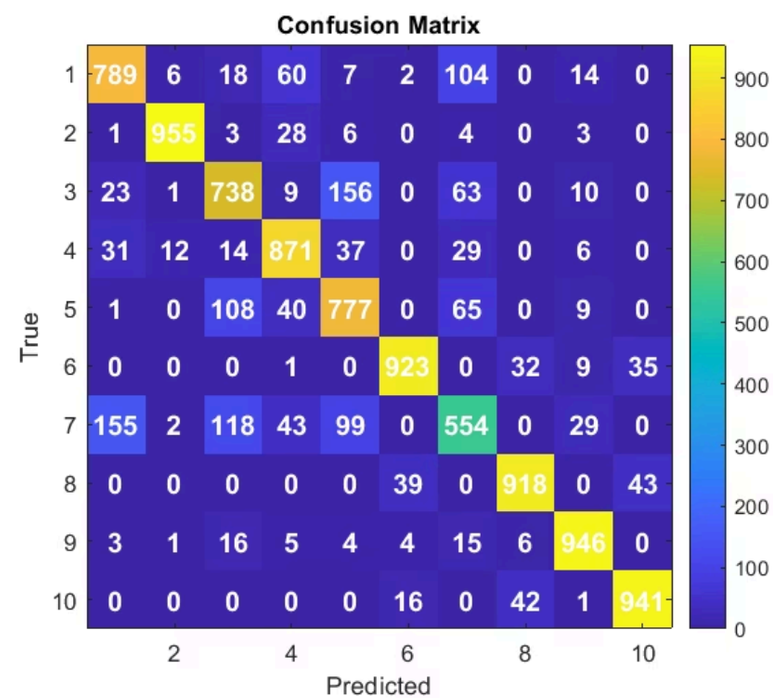
Made with GAMMA

Hybrid Version (based on loss equality of standard and svd training)

Implemented Hybrid version of standard DNN and SVD-modified DNN.
Idea: When the difference between the SVD loss and standard loss falls below a tolerance threshold, the training switches to standard DNN training using reconstructed weight matrices ($W = U \cdot S \cdot V^T$). This allows the network to fine-tune the full weight matrices and achieve higher final accuracy.

Dataset- Fashion MNIST
Accuracy: 84.12%





Made with GAMMA

Thank you

Made with GAMMA